

6. 프롬프트 성능 최적화 전략

1. Iterative Tuning 개요

Iterative Tuning이란 프롬프트를 지속적으로 수정·개선하면서 성능을 향상시키는 반복적 설계 기법입니다. 한 번에 완벽한 프롬프트를 만드는 것은 어려우므로, 초기 프롬프트로 시작해 모델 응답을 평가한 뒤 피드백 루프를 통해 프롬프트를 조정합니다. 이러한 반복 개선 루프에서는 매 iteration(반복)마다 출력에 대한 정량적 지표(점수)를 수집하고, 이를 바탕으로 다음 프롬프트 버전을 설계합니다.

단일 설계로 끝내는 전통적인 방식은 모델 출력의 예측 불가능성으로 인해 한계가 있습니다. 반면 Iterative Tuning은 데이터 기반의 과학적 접근으로, 프롬프트 성능을 점진적으로 끌어올립니다. 예를 들어, 초기에 모델이 지시에 따르지 않거나 부정확한 응답을 했다면, 다음 반복에서 지시를 강화하거나 포맷을 수정하여 개선할 수 있습니다. 이렇게 실험과 분석을 반복함으로써 프롬프트 엔지니어링을 체계적인 최적화 프로세스로 발전시킵니다. 모델의 자체 성능에 대한 피드백을 반영하므로 "모델에게 스스로를 가르치게 하는" 효과도 얻을 수 있습니다.

Note: Iterative Tuning은 소위 **프롬프트 버저닝(prompt versioning)**이라고도 합니다. 각 프롬프트 수정안을 버전으로 관리하면서 어떤 변화가 성능 향상에 기여했는지 추적하는 것이 좋습니다. 이러한 버저닝을 통해 실험 기록을 남기고 최적의 프롬프트를 재현 가능하게 만들 수 있습니다.

2. 실습 단계

이번 실습에서는 Iterative Tuning 절차를 단계별로 수행합니다. 하나의 프롬프트에 대해 여러 차례의 개선(iteration)을 거치면서, 각 단계에서 결과를 확인하고 평가합니다. 전체 루프는 다음 네 단계로 구성됩니다:

1. **초기 프롬프트 설계 및 결과 확인:** 우선 해결하려는 과제에 대해 **기본 프롬프트**를 작성합니다. (예: 긴 글 요약하기, 질의응답 등). 이 프롬프트를 GPT-4o-mini 모델에 입력하여 **초기 응답**을 얻습니다. 초기 결과를 관찰하여 현재 성능 수준과 문제점을 파악합니다.
2. **응답 수집 및 자동 평가:** 모델의 응답을 저장하고, 사전에 정의한 **LLM 평가자**(GPT-4o-mini 활용)를 사용해 자동 평가를 수행합니다. 평가자 모델에게 **평가 기준**(예: 정확성, 형식 준수, 간결성)을 알려주고, 응답을 **JSON 형식의 점수**로 출력하게 합니다. 예를 들어 평가 결과 JSON이 `{"정확성": 7, "형식충실도": 9, "간결성": 6}` 와 같이 나오도록 합니다. 이 단계에서 각 평가 점수를 기록해둡니다.

3. **개선된 프롬프트 설계 및 반복:** 2단계 평가 결과를 바탕으로 **프롬프트를 수정**합니다. 응답의 부족한 점을 보완하기 위해 프롬프트에 **추가 지시사항을 넣거나 예시를 제공하는** 등 변화를 줍니다. 수정된 프롬프트를 다시 모델에 입력해 새로운 응답을 얻고, **다시 한번 평가자를 통해 자동 평가**합니다. 이렇게 **개선 → 실행 → 평가**의 사이클을 원하는 만큼 반복합니다. 각 반복(iteration)의 평가 점수를 이전과 비교하여 **개선 여부**를 확인합니다.
4. **반복별 평가 결과 비교 및 시각화:** 모든 반복을 마친 후, iteration별 평가 점수를 **한눈에 비교**합니다. 어느 반복에서 어떤 지표가 크게 향상되었는지, 더 이상의 개선 여지가 없는 지 등을 판단합니다. Pandas DataFrame 등에 각 iteration의 점수를 정리하고, matplotlib 등을 이용해 **막대 그래프(bar chart)**나 **선 그래프(line chart)**로 **시각화**합니다. 이를 통해 **개선 추세**를 확인하고, 최종적으로 가장 우수한 프롬프트 버전을 선택하거나 회고합니다.

3. 실습용 프롬프트 유형

이번 실습에서는 **다양한 유형의 프롬프트**에 Iterative Tuning을 적용해 봅니다. 각 유형마다 요구사항과 평가 기준이 조금씩 다르므로, **맞춤 전략**으로 접근해야 합니다:

- **요약 프롬프트:** 긴 문장을 1문장 또는 3줄 요약으로 압축하는 과제입니다. 모델이 중요한 내용을 놓치지 않고 **핵심만 간결하게 요약**했는지를 봅니다. 초기에는 불필요한 세부 정보를 포함할 수 있으므로, 반복 개선을 통해 **더 짧고 핵심적인 요약문**을 얻도록 프롬프트를 다듬습니다. 평가 기준은 **정확성(중요 내용 포함 여부)**과 **응답 간결성** 위주로 설정합니다.
- **Q&A 프롬프트:** 지식 기반 질문에 대해 정확하고 간결한 답변을 생성하는 과제입니다. 예를 들어 역사적 사실에 대한 질문에 한두 문장으로 답하게 할 수 있습니다. Iterative Tuning을 통해 **정확한 정보**가 포함되도록 하고, 불필요한 수식어나 추측을 제거하여 **명료한 답변**을 유도합니다. 평가 기준으로 **정확성(사실 여부)**과 **응답의 명료성**을 주요하게 고려합니다.
- **포맷 변경 프롬프트:** 자유형 텍스트를 특정 **구조화 형식(JSON, 표 등)**으로 변환하는 과제입니다. 예를 들어, 문단으로 된 데이터를 JSON 필드로 바꾸거나, 주어진 내용을 표로 요약하게 할 수 있습니다. 모델이 형식 요구사항을 정확히 따르는지가 핵심이므로, Iterative Tuning 과정에서 **형식 충실도**를 높이는 데 집중합니다. 필요하다면 프롬프트에 **출력 예시 템플릿**을 포함시켜 모델이 그 형식을 학습하도록 개선합니다.
- **코드 생성 프롬프트:** 자연어로 기술된 명세를 읽고 간단한 Python 함수를 작성하는 과제입니다. 예를 들어 "리스트를 입력받아 정렬된 리스트를 반환하는 함수 작성" 같은 요청을 프롬프트로 줍니다. 초기 출력에서 문법 오류나 요구사항 미충족이 있을 수 있으므로, 반복적으로 **지시사항을 세분화**하거나 **예시 입력/출력**을 제공하여 정확한 코드를 얻

도록 합니다. 평가 기준은 **정확성(요구 기능 구현 여부)**, **문법 적합성(실행 가능 코드인지)**, **최적의 코드 품질(간결하고 효율적인 코드)** 등을 사용할 수 있습니다.

이러한 각 유형별 프롬프트에 대해, Iterative Tuning을 수행하면서 **출력 품질이 어떻게 향상되는지 직접 확인**하게 됩니다. 유형에 따라 어떤 전략이 가장 효과적인지도 비교해 볼 수 있습니다.

4. 평가 기준 및 LLM 평가자 설정

평가 기준은 모델 응답의 품질을 판단하는 척도로, Iterative Tuning의 **나침반 역할**을 합니다. 이번 실습에서는 아래와 같은 기준을 활용하여 자동 평가를 실시합니다:

- **정확성:** 응답 내용이 사실에 부합하고, 질문이나 지시에 정확하게 답했는지를 평가합니다. 예를 들어 Q&A에서는 정답률에 해당하고, 요약에서는 원문의 핵심 정보가 정확히 담겼는지를 의미합니다.
- **형식 충실도:** 출력이 요청된 **형식이나 구조를 제대로 따랐는지**를 평가합니다. 예를 들어 JSON 포맷으로 요청했는데 추가 설명이 붙거나 형식이 어긋나지는 않았는지, 표로 요청했을 때 정확한 표 구조를 따랐는지 등을 확인합니다.
- **응답 간결성:** 불필요하게 장황하지 않고 **간결하게 핵심만 전달했는지**를 봅니다. 특히 요약이나 Q&A 응답의 경우 너무 길게 늘어놓지 않고 요구된 분량에 맞게 잘 축약되었는지 평가합니다.
- (추가 가능 기준) **창의성/완전성:** 과제에 따라 창의적인 표현이 필요한 경우는 **창의성**을, 코드 생성 등에서는 요구된 모든 기능을 다 구현했는지 **완전성**을 평가할 수도 있습니다.

각 평가 기준별로 **점수 범위**를 정의합니다. 예를 들어 110 점 척도 또는 15 점 척도로 평가할 수 있습니다. 또는 “만족/부분만족/불만족” 같이 등급을 줄 수도 있지만, **수치화된 점수**가 반복 개선 추세를 보기에는 용이합니다.

- **LLM 평가자(GPT-4o-mini)**는 미리 설정된 프롬프트를 통해 모델 응답을 채점하도록 합니다. 즉, 평가자 역할의 별도 프롬프트를 만들어, 수강생의 응답과 이상적인 응답 기준을 비교하여 **JSON 형태의 평가 결과**를 출력하게 합니다. 예를 들어 평가자 프롬프트에는 다음과 같은 지시를 줄 수 있습니다:

“다음은 학생 모델의 출력과 평가 기준입니다. 출력에 대하여 정확성, 형식충실도, 간결성을 1~10으로 평가하고 JSON으로 제공합니다. 정확성은 사실/지시와 일치 여부, 형식충실도는 요청된 출력형식 준수 여부, 간결성은 불필요한 내용 없이 요점을 담았는지를 판단합니다.”

이처럼 명시적으로 평가 기준을 설명하고 **반드시 JSON으로만 답하도록** 지시하여, 평가자가 일관된 형식으로 채점 결과를 주도록 합니다. GPT-4o-mini는 경량화 모델이지만 이러한 단순 채점 임무를 충분히 수행할 수 있으며, 필요 시 **더 큰 모델(GPT-4o)**로 평가자를 교체하여 보다 정확한 평가를 기대할 수도 있습니다.

자동 평가 결과가 나오면, 이를 **Pandas DataFrame** 등에 누적하여 iteration별로 관리합니다. Iterative Tuning의 유효성은 결국 점수 향상으로 증명되므로, 신뢰할 만한 평가자 세팅이 중요합니다. 또한 가능하면 **사람이 일부 결과를 확인하여 평가자의 판단을 검증**하는 것도 추천합니다 (예: 첫 iteration과 마지막 iteration 출력을 직접 검토해 점수가 타당한지 확인).

5. 반복 개선 전략

프롬프트를 개선할 때는 **출력 결과의 부족한 점을 메우는 방향**으로 전략을 세워야 합니다. 한번의 수정으로 모든 문제가 해결되지 않을 수 있으므로, **다양한 기법을 순차적으로 적용**하며 모델의 변화를 관찰합니다. 다음은 대표적인 **반복 개선 전략**들입니다:

- **지시사항 명확화:** 모델이 따르지 않은 지시가 있었다면 프롬프트 문구를 더 분명하게 다듬습니다. 예를 들어 “한 문장으로 답하라”는 지시를 어겼으면 **“답변은 하나의 문장으로만 작성하세요.”**처럼 강조하고, 필요한 경우 **“불필요한 설명을 하지 마세요”** 등의 제한을 추가합니다. 지시를 구체적으로 명시할수록 모델의 **주의 집중**을 도울 수 있습니다.
- **예시 추가 (Few-shot):** 모델이 원하는 형식을 잘 이해하지 못하거나 잘못된 내용을 산출한다면, **예시를 프롬프트에 포함**시켜 가이드합니다. 예를 들어 JSON 포맷 출력이 목표라면 **올바른 JSON 예제**를 보여주고 나서 새로운 데이터를 같은 형식으로 출력하게 유도합니다. 또는 Q&A 응답에서 정보 누락이 있다면, **유사 질문과 모범 답안** 예시를 제시하여 모델이 답변 방식을 학습하도록 합니다. 단, 예시를 너무 많이 주면 **컨텍스트 길이** 한계를 초과할 수 있으니 적절히 조절합니다.
- **출력 패턴/형식 조정:** 프롬프트에 요구하는 **출력 패턴**을 명시적으로 지정합니다. 예를 들어 “응답을 JSON으로 출력하되, `{"답변": "..."}` 형태로만 작성” 또는 “표 형태로, 컬럼은 X, Y, Z로 구성”처럼 원하는 구조를 상세히 적어줍니다. 초기 프롬프트에서 형식 지시가 부족했다면 반복 개선 시 이를 추가합니다. 또한 필요하다면 **역할 시스템 메시지**(예: “당신은 깐깐한 문법 검사관입니다...””)를 사용해 특정 말투나 어조, 역할로 답하게 유도할 수도 있습니다.
- **출력 오류 수정:** 모델 응답에 사실 오류나 논리적 모순이 발견되면, 해당 내용을 프롬프트에 **반영하여 수정**합니다. 예를 들어 모델이 어떤 개념을 잘못 설명했다면, 다음 iteration에서는 **“올바른 정의: ~~”** 같이 정보를 제공하거나 **“잘못된 정보를 포함하지 말 것”**이라고 명시합니다. 이처럼 이전 출력의 오류를 피드백으로 활용하면 모델이 같은 실수를 반복하지 않도록 유도할 수 있습니다.

- **Chain-of-Thought 유도:** 복잡한 문제에 대한 추론이 부족하다면, 프롬프트에 **체인 형태의 사고를 유도하는** 문구를 추가할 수 있습니다. 예를 들어 **“문제를 풀기 전에 단계별로 생각을 정리하세요”** 라고 지시하여, 모델이 중간 과정을 더 신중히 거치도록 유도합니다. 다만 최종 답변에는 그 추론 과정을 제외하거나 포함할지 여부도 지시해야 합니다. 이 기법은 논리적 정확성과 일관성을 높이는 데 도움이 됩니다.
- **모델 파라미터 조정:** 주로 프롬프트 문구 수정으로 해결하지만, 필요한 경우 **LLM의 매개변수도** 조절해볼 수 있습니다. 예를 들어 응답의 **일관성**을 높이고 싶다면 *temperature* 값을 낮춰서 모델의 출력을 더 결정론적으로 만들 수 있습니다. 반대로 **창의적 아이디어**를 더 끌어내야 하는 과제라면 *temperature*를 약간 높입니다. 이 또한 Iterative Tuning의 한 부분으로서, 동일 프롬프트라도 파라미터 변화에 따른 결과 차이를 관찰하여 최적 조합을 찾습니다.

위의 전략들을 한꺼번에 적용하기보다는, **한 번에 하나씩 또는 순차적으로 적용**하며 어떤 변화가 효과적이었는지 파악하는 것이 중요합니다. 예를 들어 2번째 iteration에서는 지시사항만 보강해 보고, 3번째 iteration에서는 그 상태에서 예시를 추가해보는 식입니다. 이렇게 해야 각 조치가 성능에 미치는 영향을 **분석**할 수 있고, 최종 프롬프트 설계에 유용한 인사이트를 얻습니다.

또한 Iterative Tuning 과정에서 **모델 스스로에게 피드백**을 요청하는 기법도 있습니다. 예를 들어, 모델 응답 후에 **“위 답변에서 개선할 점이 무엇인지 생각하고 답변을 수정하세요”**라고 추가 프롬프트를 주어 **Self-Refinement**을 시도할 수도 있습니다. 이는 Iterative Tuning을 모델 내부 프로세스로 수행하게 하는 실험적 방법이며, 필요시 GPT-4o와 같은 상위 모델에게 **프롬프트 개선 제안**을 받아볼 수도 있습니다. 다만 이번 강의의 초점은 **사용자에 의한 프롬프트 개선**이므로, 모델 피드백은 보조적인 참고용으로만 다룹니다.

6. 평가 결과 시각화 및 분석

반복 실험을 통해 얻어진 **평가 점수 데이터**를 토대로, 프롬프트 개선 효과를 시각적으로 분석합니다. 각 iteration별 점수를 정리한 표를 만들고, 이를 **그래프로 그려 추세를 파악**합니다. 시각화를 통해 얻을 수 있는 인사이트는 다음과 같습니다:

- **반복 횟수에 따른 점수 추이:** iteration을 거듭할수록 점수가 상승하는지 (개선되는지) 확인합니다. 예를 들어 초기 정확성 6점이 iteration 3에서 9점으로 올랐다면, 해당 프롬프트 수정이 효과적이었음을 보여줍니다. 점수가 **정체**되는 구간이 있다면 더 이상의 큰 개선이 어려운 **한계점**에 도달했을 가능성을 시사합니다. 이럴 때는 다른 전략을 시도하거나 해당 프롬프트의 최적 버전을 선정할 수 있습니다.
- **지표별 세부 분석:** 정확성, 형식충실도, 간결성 등 **각 지표별로 별도의 그래프**를 그려볼 수 있습니다. 어떤 지표는 크게 개선되었지만 다른 지표는 오히려 악화되지는 않았는지 살펴봅니다. 예를 들어 형식충실도를 높이려다 보니 응답이 지나치게 길어져 간결성이

떨어지는 trade-off가 있었는지 확인합니다. 이를 통해 **프롬프트 수정의 균형**을 맞추는 법을 배웁니다.

- **프롬프트 유형별 비교:** 여러 유형의 프롬프트 실험 결과를 한 그래프에 나타내어 **최적화 효과를 비교**합니다. 예를 들어 요약 프롬프트와 코드 생성 프롬프트의 점수 향상 폭을 비교하면, 어떤 과제가 **LLM 개선에 더 민감**한지 알 수 있습니다. 각 유형별 최종 성능(최후 iteration 점수)을 비교하여 **상대적 난이도**나 **모델 한계**도 유추해볼 수 있습니다.

시각화 예시로, 아래와 같은 그래프를 생각해볼 수 있습니다:

- **Iteration별 평균 점수 추이 (Line Chart):** x축은 iteration (1, 2, 3, ...), y축은 세 평가 기준의 **평균 점수**입니다. 각 iteration에서 모든 과제 유형에 대한 평균 혹은 전체 점수의 평균을 계산해 선으로 연결합니다. 이 그래프는 **전반적인 성능 향상 경향**을 보여줍니다. 대부분 우상향 곡선을 그리면 성공적인 튜닝이며, 중간에 평평해지면 개선 둔화 시점입니다.
- **프롬프트 유형별 최종 점수 (Bar Chart):** x축은 프롬프트 유형(요약, Q&A, 포맷, 코드), y축은 해당 유형의 **초기 점수와 최종 점수**를 나란히 막대로 표시합니다. 각 유형마다 다른 색깔의 막대 두 개(iteration 1 vs Last iteration)를 그려 **개선 전후 비교**를 명확히 합니다. 이를 통해 어떤 유형에서 **점수 상승 폭**이 컸는지, 최종적으로 **절대 점수**가 높은지 등을 파악합니다.

시각화는 Python의 **Pandas**와 **Matplotlib** 라이브러리를 활용합니다. 예를 들어, 각 iteration별 평가 결과를 모아 DataFrame으로 만든 다음, **groupby** 로 iteration별 평균을 구하거나 **pivot** 으로 유형별 점수를 정리할 수 있습니다. 그런 후 Matplotlib으로 막대그래프를 그리면 됩니다. 간단한 예시 코드 스니펫은 다음과 같습니다:

##실습 6.1. Google Colab 실습

```
import pandas as pd
import matplotlib.pyplot as plt

# iterations_results는 각 반복에 대한 평가 점수를 담은 리스트 예시
df = pd.DataFrame(iterations_results)
# iteration별 정확성 평균 추출 예시
mean_by_iter = df.groupby('iteration')['정확성'].mean()

plt.figure(figsize=(6,4))
plt.plot(mean_by_iter.index, mean_by_iter.values, marker='o')
plt.title('Iteration별 정확성 평균')
plt.xlabel('Iteration')
```

```
plt.ylabel('정확성 평균 점수')
plt.show()
```

위 코드는 iteration별 정확성 평균 추이를 선 그래프로 그리는 예시입니다. 이와 유사하게 형식충실도, 간결성도 동일하게 그리거나, `plt.bar` 등을 이용해 막대그래프를 작성할 수 있습니다. 실제 실습에서는 여기서 얻은 `iterations_results` 데이터를 가공하여 다양한 그래프를 시도해보세요.

시각화 결과를 보면, 숫자 표만 볼 때보다 개선 효과를 직관적으로 이해할 수 있습니다. 또한 그래프를 보고 토론하면서 어떤 iteration에서 어떤 변화가 주요한 개선을 이끌었는지 분석해보는 것도 좋습니다. 이러한 데이터 중심의 해석은 추후 새로운 프롬프트를 설계할 때도 큰 도움이 될 것입니다.

7. Colab 실습 환경 설정

이번 실습은 Google Colab 노트북에서 Python을 사용하여 진행됩니다. 원활한 실습을 위해 **OpenAI API 연동 및 환경 설정**을 먼저 확인해야 합니다:

- **API 키 설정:** OpenAI API를 사용하기 위해 개인 API 키를 Colab에 등록합니다. Colab의 `google.colab` 패키지를 활용하면 시크릿 키를 안전하게 불러올 수 있습니다. 아래 코드로 저장된 키를 가져와 변수에 넣습니다:

##실습 6.2. Google Colab 실습

```
from google.colab import userdata
import os

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')
```

위 코드 실행 후 `API_KEY` 변수가 본인의 OpenAI API 키 문자열로 설정되었는지 확인합니다. 키가 없다면 OpenAI 계정에서 발급받아 Colab에 등록해야 합니다.

- **OpenAI 라이브러리 업데이트:** 원활한 사용을 위해 **OpenAI Python 라이브러리 버전 1.xx**가 요구됩니다. Colab에 기본 설치된 버전을 확인하고, 낮다면 `!pip install --upgrade openai` 로 최신 버전으로 업그레이드합니다. 버전 1.35.14부터 GPT-4o, GPT-4o-mini 등의 신규 모델을 지원하거나 개선된 기능이 포함되어 있으므로 업데이트가 중요합니다.
- **클라이언트 생성 및 요청:** Python에서 OpenAI API 호출을 위해 `openai` 패키지를 import하고 API 키를 설정합니다. 이후 ChatCompletion 클라이언트를 생성하여 프롬프트를 보내고 응답을 받을 수 있습니다. 예시 코드는 다음과 같습니다:

```

import openai

client = openai.OpenAI()

# GPT-4o-mini 모델을 사용한 예시 대화 요청
messages = [
    {"role": "system", "content": "너는 도움말을 제공하는 어시스턴트야."},
    {"role": "user", "content": "이 문장을 한글로 요약해 줘: ..."}
]
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    temperature=0
)
print(response.choices[0].message.content)

```

위 예에서는 대화 형식으로 system 역할과 user 역할 메시지를 정의하고, `chat.completions.create` 를 통해 GPT-4o-mini 모델에 요청했습니다. `temperature=0` 은 출력을 **deterministic**하게 하기 위해 설정한 값입니다. 응답 객체의 `choices[0].message.content` 에 **모델의 답변 텍스트**가 들어있으므로 이를 출력하거나 변수에 저장할 수 있습니다.

- **평가자 호출:** 프롬프트 평가를 위해 GPT-4o-mini를 **평가자 모드**로 다시 호출합니다. 평가자 프롬프트 (앞서 정의한 기준과 출력 형식 지시)를 system 또는 user 메시지로 설정하고, 모델의 이전 응답을 이에 포함시켜 `chat.completions.create` 를 수행합니다. 모델이 JSON 평가 결과를 반환하면 `json.loads()` 등을 통해 Python 객체로 파싱하여 다룹니다. 이때도 `response.choices[0].message.content` 로 결과 텍스트를 추출합니다.
- **반복 자동화(Optional):** 숙련된 수강생의 경우 위 과정(모델 응답 생성 → 평가자 평가 → 결과 저장 → 프롬프트 수정)을 **for loop**로 자동화해볼 수 있습니다. 예를 들어 프롬프트 목록이나 파라미터 변경 목록을 만들어 순차적으로 모델을 실행하고, 결과를 리스트에 누적한 뒤 DataFrame으로 만드는 방식입니다. 이렇게 하면 여러 iteration을 손쉽게 실험할 수 있지만, 우선은 **수동으로 한 단계씩 확인하면서 진행**하는 것을 권장드립니다 (모델 예측 특성을 이해하는 학습 효과를 위해).

Colab 환경에서 실행 중 문제가 생기면 세션 재시작이나 API 키 재확인, 또는 openai 패키지 재설치 등을 시도해야 합니다.